

TALLER EVALUATIVO 2: ORQUESTACIÓN CI/CD CON JENKINS, EMPAQUETAMIENTO INMUTABLE EN NEXUS Y WEBHOOKS CON SMEE.IO

Ingeniería de Software V

FACULTAD DE INGENIERÍA, DISEÑO Y CIENCIAS APLICADAS
INTELIGENTES

DEPARTAMENTO DE COMPUTACIÓN Y SISTEMAS

INFORMACIÓN DE LA ACTIVIDAD

IDENTIFICACIÓN

CÓDIGO:	09776 - TIC	PERÍODO:	202620 (Semestre 2026-2)
NRC / GRUPO:	11005 / Grupo 001	MODALIDAD:	Individual
NRC / GRUPO:	11005 / Grupo 005	PONDERACIÓN:	15 % (Taller Evaluativo 2)
POLÍTICA IAG:	Nivel 3: Colaboración Asistida	CALIFICACIÓN:	_____ / 5.0
ESTUDIANTE:	_____	CÓDIGO:	_____

1 Contexto del Problema & Ecosistema CD On-Premise

1.1 Descripción del Desafío de Ingeniería DevOps

La organización **Icesi EduTech** requiere modernizar **StudyTrack**, su plataforma de seguimiento de tareas académicas, compuesta por dos componentes desacoplados:

- Backend REST** (`studytrack-api`): Microservicio en **Spring Boot 3 (Java 17)** con JPA y base de datos H2 en memoria, encargado de exponer operaciones CRUD de tareas académicas (`GET, POST, PUT, DELETE, PATCH /api/tasks/{id}/complete`).
- Frontend SPA** (`studytrack-frontend`): Interfaz de usuario en **React 18 + Vite + TypeScript** que consume el API a través de la variable de entorno `VITE_API_URL`.

A diferencia del Taller 1 (donde la automatización residía en GitHub Actions, un servicio administrado en la nube), en este taller el estudiante debe desplegar y operar **su propio ecosistema de integración continua en modalidad on-premise**: un controlador **Jenkins** en contenedor, un repositorio de artefactos **Sonatype Nexus** para empaquetamiento inmutable, y un relay **Smee.io** que reenvía los webhooks de GitHub hacia Jenkins sin necesidad de exponer puertos a Internet. El objetivo es comprender el costo operativo y el valor de gobernanza de administrar la propia infraestructura de CI/CD, en contraste con un servicio SaaS.

Componente	Tecnología & Stack	Puerto / Endpoint
Backend REST	Java 17, Spring Boot 3.2, Spring Data JPA, H2	Puerto 8080 (<code>/api/tasks</code>)
Frontend SPA	React 18, TypeScript, Vite 5, Nginx Alpine	Puerto 3000 (Proxy: <code>/</code>)
Orquestador CI	Jenkins LTS (JDK 17), pipeline declarativo (<code>Jenkinsfile</code>)	UI: 9182 • Agentes: 50000
Repositorio de Artefactos	Sonatype Nexus 3 (<code>maven-releases, docker-hosted</code>)	UI: 9181 • Docker Registry: 9180
Relay de Webhooks	<code>smee-client</code> (canal público en <code>smee.io</code>)	Sin puertos expuestos (saliente HTTPS)

Nota: los puertos anteriores son sugeridos para evitar colisiones al ejecutar todo el ecosistema en una sola máquina (la del estudiante). Usted deberá validarlos y ajustarlos en los archivos `.env` entregados si detecta un conflicto local.

2 Procedimiento Experimental & Tareas a Desarrollar

2.1 Fase 1: Pruebas Unitarias y Compilación de Artefactos (20 min)

En el código base suministrado en `codigo_base/`:

- En `backend/`, ejecute la suite de pruebas unitarias y de controlador con Maven. Verifique que `TaskServiceTest` y `TaskControllerTest` reporten 100 % de éxito.
- En `frontend/`, valide la consistencia de tipos TypeScript y genere el build de producción.

2.2 Fase 2: Empaquetamiento Inmutable Multi-Stage (30 min)

Complete los archivos `Dockerfile` del código base aplicando buenas prácticas de reducción de peso, seguridad e **inmutabilidad de artefactos** (queda terminantemente prohibido el uso del tag `:latest`):

- **Backend** (`backend/Dockerfile`): *Stage 1 (Builder)* `maven:3.9.6-alpine` compilando con `mvn package -DskipTests`; *Stage 2 (Runtime)* `eclipse-temurin:17-jre-alpine`, copia del `.jar`, usuario no root (`appuser`) y puerto 8080. Complete además el bloque `<distributionManagement>` del `pom.xml` apuntando al repositorio `maven-releases` de su Nexus local.
- **Frontend** (`frontend/Dockerfile`): *Stage 1 (Builder)* `node:20-alpine` (`npm install + npm run build`); *Stage 2 (Runtime)* `nginx:alpine`, copia de `dist/` y complete el `nginx.conf` con soporte de *fallback* SPA (`try_files`).
- Defina un esquema de **versionado inmutable** para las imágenes y el JAR (por ejemplo, número de build combinado con los primeros caracteres del commit de Git) y publique manualmente una primera vez para validar credenciales.

2.3 Fase 3: Orquestación del Ecosistema CI/CD con Docker Compose (25 min)

Complete los archivos incompletos en `infra/nexus_config/` e `infra/jenkins_config/`:

- En `nexus_config/docker-compose.yml`: declare el puerto de la UI/REST, el puerto del Docker Registry hosted y el *health-check* sobre `/service/rest/v1/status`.
- En `jenkins_config/docker-compose.yml`: declare la red compartida (`cicd_network`), el volumen persistente `jenkins_home`, el montaje del socket de Docker, y el servicio `smee` con `depends_on`: `jenkins` y la variable `SMEE_URL`.
- Levante ambos stacks y obtenga la clave inicial de Jenkins.
- Cree un canal en <https://smee.io>, configúrelo como `SMEE_URL` en su `.env`, y registre ese mismo canal como Webhook (Payload URL) en la configuración de su repositorio de GitHub.
- En Jenkins, instale los plugins de `jenkins_config/plugins.txt`, y cree las credenciales `nexus-credentials` (usuario/clave de Nexus) y `github-credentials` (token de GitHub).

2.4 Fase 4: Pipeline Declarativo en Jenkins (35 min)

Complete el archivo `Jenkinsfile` en la raíz del repositorio, estructurado en las siguientes etapas:

- Checkout & Test:** Clonado del repositorio y ejecución de `mvn test` sobre el backend.
- Package & Tag Immutable:** Compilación del `.jar` y construcción de las imágenes Docker multi-stage, etiquetadas con el esquema definido en la Fase 2 (número de build + hash corto del commit).
- Publish to Nexus:** Publicación del `.jar` (`mvn deploy`) y de las imágenes Docker (`docker push`) en su Nexus local, usando las credenciales configuradas en la Fase 3.
- Deploy & Smoke Test:** Despliegue local con `docker compose -f deploy/docker-compose.yml up -d` referenciando la imagen recién publicada, seguido de una validación `curl` contra `/api/tasks` con reintentos.

Realice un `git push` a su repositorio y verifique que `Smee.io` dispare automáticamente la ejecución del pipeline en Jenkins, finalizando en estado **verde (Success)**.

3 Cuestionario de Análisis DevOps (10 min)

Responda brevemente a las siguientes preguntas con base en lo desarrollado durante este taller:

1. **CI/CD Auto-hospedado vs. Servicio Gestionado:** Desde una perspectiva de arquitectura y operación, ¿qué ventajas y responsabilidades adicionales introduce mantener Jenkins, Nexus y el relay de webhooks bajo administración propia, en comparación con utilizar una plataforma CI/CD gestionada como GitHub Actions? Considere aspectos como control, disponibilidad, mantenimiento, seguridad y costos operativos.
2. **Trazabilidad y Gobernanza de Artefactos:** ¿Por qué la separación entre el código fuente, el proceso de construcción y el repositorio de artefactos es importante para garantizar la trazabilidad y reproducibilidad de una entrega? Explique cómo la relación entre commit, número de build, imagen Docker y artefacto Maven contribuye a identificar exactamente qué versión fue construida y desplegada.

4 Bitácora de Colaboración con IAG

Bitácora de Prompts y Reflexión sobre uso de IAG

De acuerdo con la **Política IAG Nivel 3: Colaboración Asistida**, escriba un documento que contenga:

- Una **bitácora de prompts** con la herramienta de IA generativa utilizada durante el taller, indicando en qué fase se usó y qué tipo de ayuda solicitó (explicación, depuración, generación de configuración, etc.).

5 ENTREGABLES

INSTRUCCIONES DE ENTREGA

1. **Repositorio en GitHub:** Subir el código base con las soluciones implementadas (incluyendo el `Jenkinsfile` en la raíz) y verificar que la última ejecución del pipeline en Jenkins pase en **Success**.
2. **Documento de Resultados (.PDF o .MD) Debe pushear el documento en una carpeta docs dentro del repositorio.**
 - Respuestas al cuestionario técnico (Sección 3)
 - Bitácora de prompts IAG.

6 Rúbrica de Evaluación Modularizada por Punto (Escala 0.0 a 5.0)

Punto Entregable	Pts	Excelente (4.5 – 5.0)	Aceptable (3.0 – 4.4)	Insuficiente (0.0 – 2.9)
2.1–2.2 Empaquetamiento Inmutable	0.8	Multi-stage impecable en ambos servicios, usuario no root, versionado semántico atado al commit, sin uso de :latest.	Multi-stage funcional pero usa imagen pesada, ejecuta como root o versiona de forma inconsistente.	Dockerfile de una sola etapa, uso de :latest o imagen de tamaño excesivo.
2.2 Publicación en Nexus	0.8	JAR e imagen Docker publicados correctamente en maven-releases y docker-hosted, credenciales gestionadas de forma segura.	Publica en Nexus pero con errores menores de configuración (repositorio incorrecto, credenciales en texto plano).	No logra publicar artefactos o publica en un repositorio público en lugar de Nexus.
2.3 Orquestación de Infraestructura CI/CD	0.7	Stack Jenkins + Nexus + Smee levantado sin errores, healthchecks, red y volúmenes correctamente declarados.	Stack levanta pero sin healthchecks o con dependencias mal declaradas.	Error al levantar el stack, conflicto de puertos o redes.
2.4 Pipeline Declarativo Jenkins	1.5	Jenkinsfile con las 4 etapas completas, ejecución en verde (<i>Success</i>), artefactos versionados correctamente en cada build.	Pipeline funcional pero con etapas redundantes, sin manejo de fallos o con pasos manuales innecesarios.	Pipeline falla, no ejecuta todas las etapas o sintaxis de Jenkinsfile incorrecta.
2.3–2.4 Integración de Webhook (Smee.io)	0.5	El push a GitHub dispara automáticamente el build vía Smee sin intervención manual.	El webhook funciona pero requiere reintentos o configuración manual adicional.	El webhook no funciona; el estudiante ejecuta el build manualmente sin justificar la causa.
3–4 Cuestionario, Autorreflexión y Bitácora IAG	0.7	Respuestas técnicas rigurosas y reflexión genuina sobre el valor de Jenkins/Nexus, bitácora de prompts completa y bien documentada.	Respuestas breves o bitácora de prompts incompleta.	Respuestas erróneas conceptualmente, sin autorreflexión o sin bitácora IAG.
CALIFICACIÓN TOTAL		5.0 Nota para aprobar ≥ 3.0		